Oblsk Negotiation Agent — Codebase Overview

A Python agent that prices every creator deal and negotiates a thread on your behalf, gated by configurable human approvals. Each outbound message waits for a person to sign off until autonomy is switched on. There's a real-world example campaign in `examples/`, an interactive demo (`demo.py`), an HTTP service for the Oblsk platform (`service.py`), a chat REPL, a replay tool that compares the agent against history, and a sparring tool that lets Claude play the other side.

The system is **two halves that meet at one handoff**: every dollar the negotiator commits comes out of the calculator, and the negotiator never invents a price.

The two halves

1. The calculator (the math side)

- Fits a **heavy-tailed model** of a creator's per-video views. Creators' averages hide the occasional viral hit that drives real value, so views are fit with a lognormal body spliced onto a Pareto tail when excess kurtosis in log-space warrants it, or with a recency-weighted empirical prior when history is too thin. The same draws power both single-format and multi-format deals.
- Runs a **Monte Carlo** over that model: each draw samples views for every video, turns them into attributed revenue at the campaign's `revenue_per_view`, and reports the distribution (p10 downside, winsorized mean expected, p90 upside).
- Builds the **pricing ladder** from the distribution:
  - **anchor** — where to OPEN: low but credible, leaves room to move up
  - **target** — where to AIM: hits the ROI goal at expected delivery
  - **walk_away** — the CEILING: the p10 downside still clears the minimum ROI (or a CPM cap, whichever binds first)
- Includes an **authenticity heuristic** (bought-follower / engagement-pod checks) that becomes a downside risk discount and tightens the ceiling automatically, plus a placeholder **CPM benchmark table** by platform and follower tier (calibrate before trusting absolute dollars).

2. The negotiator (the conversation side)

- A **behavior tree** — one pass per inbound message, top-to-bottom, first guard that fires wins.
- The branch **order is data**: it lives in `tree_spec.yaml` as ordered `(guard, handler)` names. Adding a branch is a config edit plus its guard and handler; the diagram in `figures/` is drawn from the same order.
- The branches cover, in order: human stepped in → creator accepted → questions → contract terms (always escalate — the agent never negotiates paper) → references to a call the agent was not on → wants a call (no counter) → no offer yet (open with anchor) → follow-up

with no counter (hold firm, do not bid against yourself) → rejected twice with no counter (soft close) → their ask is fine for us (accept) → their ask is wildly high (escalate) → out of rounds → flat rejected (revise once toward target) → revised flat also rejected (escalate to bundle) → bundle on the table (add sweetener or escalate).

The decision goes to **prose** — a real email the creator sees. Prose is **LLM-first** (Claude drafts a short, human message from the decision plus recent thread) with a deterministic template fallback. A draft that drops the calculator's numbers is discarded, because a rewording that loses the price is worse than a template that keeps it.

The package layout (`oblsk_negotiator/`)

Module	What it does
`ev_engine.py`	View-model fit plus Monte Carlo EV. Defines the single-format `Deal` and the multi-format `Bundle` (with a cost-mirroring surface so the rest of the agent treats them interchangeably), plus the `FormatCatalog` (the value side).
`pricing.py`	The three-rung ladder (`price_ladder`), the authenticity heuristic (`authenticity`), and the CPM benchmark table.
`bundles.py`	Pure deal-building primitives: `flat_offer` (the opening), `bundle_from_flat` (more videos at a fair per-video rate), `add_non_price_sweetener` (faster pay, usage rights, whitelisting), and `compose_bundle` (composes a multi-format package from a rate card + catalog that already clears the ROI hurdle).
`rate_card.py`	The creator's asking side: canonical formats, **market-measured** bundle discounts (combo ~10%, syndication ~15%, volume ~10%), and a rate-card parser.
`behavior_tree.py`	The decision logic — guards + handlers as registries, the `DecisionContext`, and `decide()` which walks the spec.
`tree_spec.yaml`	The branch order — config-only. The diagram in `figures/` mirrors it.
`qa.py`	Answers questions from the campaign brief (LLM-first), with its own fallback and a 3rd-question-nudges-to-offer rule.
`prose.py`	Decision → human email. LLM draft, template fallback, numbers-preserved check.
`llm.py`	One wrapper around the Anthropic SDK with an offline fallback; respects `OBLSK_NO_LLM=1`.

Module	What it does
`campaign.py`	One YAML per campaign: brief + economics + negotiation stance + `us_aliases` (for thread recognition). Swap campaigns by swapping files.
`state.py`	One DB row per thread — `NegotiationState`. Tracks phase, rounds, approvals, autonomy level, concession history, detects when a human stepped in. JSON-serializable for pause/resume.
`events.py`	Append-only event log mirror of state (`MessageReceived`, `Decision`, `Approval`, `Sent`, `HumanRejected`) plus `fold_events` (replay log → state). State is reconstructable from the log.
`service.py`	The HTTP `/suggest` endpoint the Oblsk platform calls. Stateless — state is derived from the thread itself, so a person-made offer mid-thread is treated as the agent's position. Details in `docs/INTEGRATION.md`.
`replay.py`	Shadow-runs the agent over a real Gmail thread so you can compare what the agent would have said vs. what your team actually sent. Nothing is ever sent.
`spar.py`	Has Claude role-play the creator's manager and judge the transcript. Needs `ANTHROPIC_API_KEY`; no offline fallback (an offline counterpart would just be the SimCreator again).
`creator_sim.py`	A parameterized simulated creator with a private playbook, used for closed-loop `run_negotiation` testing without a network.
`runner.py`	The orchestrator: the loop, the approval gate (each outbound waits for `approver` or auto if below `auto_send_dollar_ceiling`), and aggregate metrics.

Tests, examples, and docs

- **`tests/`** — 72 checks, all offline. Includes replay parity, tree parity against a `tree_spec.yaml` snapshot, golden-decision regression, and the notebook smoke test.
- **`examples/unest_campaign.yaml`** — annotated real campaign config (brief, economics, full negotiation stance, `us_aliases`).

- **`examples/unest_thread.txt`** — a real agency-run negotiation the agent replays: opens within $100 of the team's opener, accepts the same $2,000/video, holds firm on follow-ups, proposes a call when the manager confirms availability, and escalates the contract-revision email to a human.
- **`notebooks/`** — interactive walkthrough of the same ground.
- **`docs/USAGE.md`** — the full plain-language manual (install, every way to run it, every feature explained).
- **`docs/CALCULATOR.md`** — the pricing model in math plus intuition.
- **`docs/INTEGRATION.md`** — how the Oblsk platform calls `/suggest`, plus the response contract.

How it runs

1. **Fit** the creator's view model from `recentPostViews` (auto-fallback to a tier prior and scale it when too few posts are available; recency-weighting and sponsored haircut configurable).
2. **Build** the pricing ladder under the campaign's policy (ROI target / ROI min / CPM cap / anchor factor / risk discount).
3. **Read** the inbound: classify intent (LLM or rules), extract the ask, detect contract-terms negotiation, call-reference, wants-call flags.
4. **Decide** by walking the behavior tree against the ladder and the thread's state.
5. **Render** the outbound: LLM draft with a template fallback; the numbers from `decision.deal` must appear in the message or the draft is discarded.
6. **Gate** through approval (auto in autonomous mode, but never above `auto_send_dollar_ceiling`).
7. **Log** to state *and* the append-only event log, so a thread can be paused, replayed, or shadow-run any time.

What's separable

- The **behavior tree is data** (`tree_spec.yaml`); the **pricing math is code** (`pricing.py`, `ev_engine.py`); the **words are LLM-drafted but number-checked**.
- That means you can tune the negotiation stance in YAML without touching pricing code, and tune prices in code without rewriting the conversation.
- Every number the negotiator uses (ROI target, ROI min, anchor factor, CPM cap, accept margin, extreme-ask multiple, bundle size, money rounding step, auto-send ceiling) lives in `CampaignContext` and is overridden per-campaign in the YAML.